

Agentic Flow

Autonomous GitHub Issue Resolution — micro1 Agentic Workflows Hackathon

Project Author: Bhavin Patel • **Repo:** github.com/bhavinmatariya/agenticflow

What This Is

Agentic Flow is a GitHub Action that turns an issue into a reviewed, tested pull request, and it won't touch a single line of code until a human has signed off on the plan. Drop one workflow file and two secrets into a repo and it just works: it goes and reads your actual codebase, comes back with one or more real fix options (never a made-up number just to look thorough), waits for you to reply in plain English, makes the edit, runs the tests, and only then opens the PR.

Built to Be Adopted, Not Copied

It's shipped as a public composite Action (**uses: bhavinmatariya/agenticflow@main**) rather than a script you'd paste into your own repo. That means anyone can point their repo at it without ever touching agentic-flow's source.

THE PROJECT IN NUMBERS

Metric	Value
Agent capabilities used	5
Real iterations shipped	22
Secrets required to adopt it	2
Estimated active developer time saved	≈ 85–95% (see Time Savings)

WHAT'S IN THIS DOCUMENT

- How the Agent Works — Full Flow: the complete 9-step pipeline, agent by agent
- How Agents Help: the five agent capabilities and why each one is there
- Baseline vs. Agentic Flow: a direct, line-by-line comparison
- Estimated Time Savings: what this actually saves a developer, by task type
- Estimated Cost Savings: time saved translated into \$ for an individual vs. a team
- Improvement Changelog: all 22 real iterations, mistakes included
- Reproduction Guide: exact setup and run steps for a clean environment
- Demo Video Script: the walkthrough used for the submission video
- Future Implementation: extending beyond GitHub Issues to Jira, Trello, and other ticket trackers

Keep Four Questions in Mind

01

Who has this problem?

Any developer or small team that triages GitHub issues by hand — reading the report, digging through the code, figuring out how to fix it, writing it, testing it, opening a PR. Solo maintainers feel this the most, because every single issue is a manual round trip before any code even gets written.

02

What bottleneck makes it worth solving?

Typing the fix was never the slow part. The slow part is understanding a corner of the codebase you didn't write, figuring out if there's an actual tradeoff worth flagging, and not breaking the app's own conventions while you're at it. Doing that carefully, on every issue, is what a small team can't keep up with.

03

Does the agent solve it well?

Agentic Flow actually goes and looks at the repo before it suggests anything. There's no hardcoded list of issue types and no fixed number of options — the model decides both, per issue. You approve with a normal comment reply. Then the implementer makes the exact edit, and a reviewer agent checks it against what you literally asked for, runs the real tests, and for UI work spins up a disposable database and a Playwright check.

04

Can another person reproduce the result?

Yes. Check the Reproduction Guide — anyone can drop the same two secrets and workflow file into their own repo and get the same pipeline. Nothing about the repo name, the issue number, or the labels is hardcoded anywhere; it all comes from the GitHub Actions event at runtime.

How Agents Help

I picked five agent capabilities for this, and I tried to use each one for a real reason rather than just checking a box.

Capability	Where It's Used	Why It Matters Here
Better context	Investigator clones the real repo, searches code, reads relevant files, and surfaces existing conventions as evidence.	Prevents proposals that violate the codebase's own established patterns.
Tools	<code>code_search</code> , <code>code_editor</code> (exact old/new-string edits), <code>test_runner</code> , <code>environment_manager</code> , <code>browser_test</code> , <code>db_verifier</code> .	Lets agents act on the real repo and verify real behavior, not just describe changes.
Memory / state	A hidden <code>agentic-flow:state</code> comment persists the approved investigation and approach across stateless GitHub Actions runs.	Each Action run is a fresh process — GitHub itself (labels + one hidden comment) is the only durable state.
Verification	Reviewer agent re-checks literal approved details against the diff, runs scoped tests, and for UI work adds a live disposable-DB + Playwright check.	Catches real defects before a human ever sees the PR.
Orchestration	Investigator → Proposer → (human approval) → Implementer → Reviewer, looping back to Implementer as needed, up to 6 rounds, each starting fresh.	Mirrors how a careful human developer actually works: propose, get sign-off, build, test, fix, repeat.

None of these five were added just to pad out the architecture. Each one is there because a specific failure mode actually showed up while I was testing — the changelog on the next few pages walks through exactly which one and why.

How the Agent Works — Full Flow

This is the complete round trip, start to finish. Every run goes through the same pipeline, whether it resolves in one pass or loops a few times before a human needs to step in.

#	Stage	Agent	What Happens
1	Issue opened	Orchestrator	A GitHub issue is opened or labeled. The workflow picks it up automatically — no manual trigger needed.
2	Investigate	Investigator	Clones the real repo, searches the codebase, reads the relevant files, and pulls out the existing conventions as evidence before anyone proposes anything.
3	Propose	Proposer	Drafts one or more genuinely distinct fix approaches based on what the Investigator found, and posts them as a plain-English comment on the issue.
4	Human replies	You	Reply on the issue in plain English — "go with option 1," "approved with changes: also make it blue," or "no, try something else."
5	Classify the reply	Response Parser	Reads the free-text reply and classifies it as approve, approve with feedback, or reject. Approve-with-feedback flows straight into the next step with the feedback attached; reject sends it back to the Proposer.
6	Implement	Implementer	Creates a branch and makes exact, surgical old-string/new-string edits — never a blind rewrite of the file.
7	Review	Reviewer	Checks the diff against every literal detail you approved, runs the tests scoped to what actually changed, and for UI work spins up a disposable database plus a generated Playwright check.
8	Verdict	Reviewer → Orchestrator	Passed → the PR is opened, linked to the issue. Needs revision → loops back to Step 6 with the specific findings attached. Stuck after 6 rounds → the issue is labeled needs-human with the full history preserved.
9	Human review	You	Normal code review and merge — the agent never merges its own PR.

Steps 6 and 7 can repeat up to 6 rounds, and each round starts a brand-new agent conversation — only a short summary of the prior attempt carries forward, so context never balloons. A hidden state comment on the issue (JSON inside an HTML comment) tracks exactly which approach was approved and what round it's on, so a "continue where you left off" reply days later resumes correctly instead of starting over. Every comment the bot posts also carries a hidden marker, so the workflow never reacts to its own comments and can't trigger itself in a loop.

Baseline vs. Agentic Flow

The baseline I compared against is about as simple as it gets: one prompt to an LLM with the issue title and body, and an instruction to “suggest a code fix for this GitHub issue.” No repo access, no tools, no verification, no human approval, no PR — just whatever the model guesses from the text alone.

Metric	Simple Baseline	Agentic Flow
Reads the real codebase	✗	✓ clone, search, read relevant files
Surfaces genuine tradeoffs, not guesses	✗ guesses	✓ dynamic, per-issue reasoning
Human approves before code changes	✗	✓ free-text comment
Makes surgical, reviewable edits	✗ would rewrite blindly	✓ exact-match old/new string edits
Runs real tests / build	✗	✓ scoped to what changed
Verifies UI behavior live	✗	✓ disposable DB + Playwright, auth-aware
Opens a PR only after passing review	✗	✓
Works on any repo via one workflow file	✗ not applicable	✓

ESTIMATED TIME SAVINGS

Formal head-to-head timing (the Evaluation run) is still pending, but this is a grounded estimate: typical manual timings for each task type, against the runtimes Agentic Flow has actually shown in testing (2–5 minutes for a single-round fix, 10–15 minutes for a multi-round change with live UI verification).

Task Type	Manual (typical)	Agentic Flow (observed)	Time Saved (est.)
Docs-only change	20–40 min	2–5 min	≈ 85–90%
Small UI fix (single component)	45–90 min	5–10 min	≈ 80–85%
Medium backend bug fix	1–3 hours	5–10 min	≈ 90–95%
Feature change with UI verification	2–5 hours	10–15 min	≈ 90%+

The time a developer spends doesn’t disappear — it moves. Instead of reading the issue, digging through the code, writing the fix, and testing it, a developer spends a few minutes reading a proposal and replying "approved." Everything after that runs unattended.

ESTIMATED COST SAVINGS

Turning the time saved into a dollar figure, at an example blended developer rate of \$50/hour (swap in your own team’s real rate — the hours saved are what actually matters).

Scenario	Issues / Mo.	Manual Hrs	Agentic Hrs	Hrs Saved	\$ Saved / Mo.	\$ Saved / Yr.
Individual developer	15	22.5	2.5	20.0	≈ \$1,200	≈ \$14,400
Small team (5 devs)	75	112.5	12.5	100.0	≈ \$6,000	≈ \$72,000

For a solo developer, that’s roughly two extra focused workdays given back every month. For a small team, it’s close to a full extra developer’s worth of monthly capacity — without hiring one.

Improvement Changelog

This is the real progression, mistakes included. I didn't clean it up to look smarter than it was.

Stage	What We Tried and Why	Evidence	Decision
Baseline	Single LLM prompt, no tools, no repo access.	Produces plausible-sounding but unverified, ungrounded suggestions.	Established the starting point.
Iter. 1–5	Built a GitHub adapter (read/write issues, labels, comments), then Investigator + Proposer agents with real code search.	Investigator found a real commented-out route and compared it to an existing /api/health pattern; Proposer produced two genuinely distinct approaches.	Kept — core investigate/propose loop works.
Iter. 6–9	Added ResponseParser (free-text approval), Implementer (surgical edits), Reviewer, Orchestrator loop, PR writeup.	End-to-end run on a real issue produced a real branch and commits; implementer hit a NoneType.message crash from a stale-SHA race condition.	Fixed with safe attribute access and auto-retry-with-fresh-fetch on conflict.
Iter. 10	Noticed the proposer suggested a native alert() as a "peer option" next to an existing custom dialog.	Codebase had zero native alert() usage anywhere — not a legitimate tradeoff.	Added a hard rule: only propose multiple approaches for genuine tradeoffs, never a convention-violating option just to manufacture choice.
Iter. 11	Improved GitHub Actions log readability.	Plain INFO module:message logs were hard to scan during live debugging.	Added retro ASCII banners, a progress bar, collapsible ::group:: sections, and a step summary report.
Iter. 12	Discovered one human comment triggered 4 workflow runs.	The bot's own posted comments re-triggered the same issue_comment webhook.	Added a hidden auto marker plus an in-app skip, later a workflow-level guard so the wasted run doesn't even start.
Iter. 13	git checkout <branch> failed with "pathspec did not match" right after a successful API commit.	Local clone was shallow (--depth 1 of main only) — the new branch was never fetched locally.	Fetch the specific branch ref before checkout, always.
Iter. 14	New-file creation via edit_file failed with 404.	Tool tried to fetch the "existing" content of a file that didn't exist yet.	Skip the fetch entirely when creating a new file.
Iter. 15	Reviewer approved a fix that didn't actually apply a requested detail ("make it blue").	Reviewer was checking "does it build," not "does it match the literal request."	Reviewer now extracts every literal requested detail and checks each individually against the diff.
Iter. 16	A malformed (non-JSON) implementer response crashed the entire run instead of retrying.	One bad model output ended a 6-round budget after 2 rounds.	Added JSON-repair retry, and wrapped each round in its own try/except so one failure doesn't kill the whole orchestration.
Iter. 17	A fresh-repo test hit a shell syntax error: unexpected token '('.	Comment body (containing quotes, parens, backticks) was interpolated directly into a run: bash string.	Moved all free-text event data into env: variables, referenced

Stage	What We Tried and Why	Evidence	Decision
			safely as "\$VAR" — also closes a script-injection risk.
Iter. 18	Same test then hit "prompt is too long: 3,685,415 tokens > 1,000,000 maximum."	Each orchestrator round was re-sending the entire raw tool-call history of all previous rounds instead of starting fresh.	Each round now starts a brand-new agent conversation; only a short summary carries forward, plus a defensive token-budget trim.
Iter. 19	A "continue where you left off" comment on a stalled issue did nothing.	The state machine only reacted to comments in the awaiting-approval label state.	Added a needs-human resume path: reads the hidden state comment, reuses the existing branch, continues instead of restarting.
Iter. 20	"Approved with changes: make it blue, separate component, dancing animation" got misclassified as revise and dead-ended with just an acknowledgment.	The response parser didn't distinguish "approve + extra detail" from a genuine rejection; the revise path was an unfinished stub.	Parser now classifies "approve with tweaks" as approve + feedback; revise loops back to the proposer with the feedback instead of dead-ending.
Iter. 21 — biggest fix	A docs-only change (one new markdown file) failed review for 6 straight rounds and never opened a PR.	Reviewer ran the whole repo's build/tests every round regardless of what changed — an unrelated npm install gap made the next build fail every time, blocking a correct doc change.	Test/build checks are now scoped to the actual changed files; the summary states plainly what was tested vs. skipped and why.
Iter. 22	Audited the approve-path and found it silently re-ran investigator + proposer after approval, matching the human's choice against a brand-new proposal.	Risk of approving "Option 2" from proposal A but implementing an unrelated "Option 2" from a freshly regenerated proposal B.	State is now captured at proposal time; approval reuses the exact investigation/approach the human actually read.
Removed	Hardcoded issue-type tags (bug/feature/etc.) considered early on.	Directly conflicts with the "full dynamic thinking" goal — the LLM should classify per issue, never from a fixed list.	Removed before it was ever built.

Reproduction Guide

If you're starting from nothing, here's exactly what to do.

SETUP

1. Fork or use any GitHub repo you want the agent to work on.
2. Add `.github/workflows/agent-flow.yml` (see repo README) — triggers on issues: opened and issue_comment: created.
3. In that repo's Settings → Secrets and variables → Actions, add ANTHROPIC_API_KEY (your own Anthropic key) and AGENT_GITHUB_TOKEN (a PAT with Contents/Issues/PRs read-write).
4. Optionally add `repos.json` at the repo root to link related repos for cross-repo issues. Not required — the agent works fine with just the primary repo.

RUNNING IT

1. Open a real GitHub issue describing a bug or feature.
2. Agentic Flow automatically investigates and posts a proposal comment within a few minutes.
3. Reply in plain English (e.g. "go with option 1" or "approved with changes: also make it blue").
4. Agentic Flow implements, tests, and either opens a PR or explains in plain language why it needs a human, listing every round it attempted and why each was rejected.

Expected runtime and cost: a single-round docs or small UI fix typically completes in 2–5 minutes and a few cents of Claude usage; a multi-round implementation with live browser verification can take 10–15 minutes.

Future Implementation

Right now Agentic Flow only understands GitHub Issues — that's where the adapter layer lives today. But the actual agent pipeline (Investigator, Proposer, Implementer, Reviewer, Orchestrator) doesn't know or care where a ticket came from. The GitHub-specific piece is a thin adapter: reading an issue, posting a comment, opening a PR. Swap that adapter for another platform's API and the same reasoning pipeline runs unchanged.

PLANNED TICKET-HANDLER ADAPTERS

- **Jira** — the most requested next step — issue + comment webhook shape maps closely to GitHub's
- **Trello** — card comments and checklists stand in for issue comments and approval replies
- **Linear** — a natural fit given its existing GitHub-style webhook and API design
- **Asana** — tasks and task comments replace issues and issue comments
- **ClickUp** — similar task/comment model, plus custom fields for approval state
- **Monday.com** — board items and updates play the same role as issues and comments
- **Azure DevOps** — work items and discussion threads, useful for teams already on Azure Repos
- **Zendesk / ServiceNow** — for teams that route engineering work through a support or ITSM queue

The near-term plan is to generalize the adapter interface first (read ticket, post comment, open change request) and ship Jira support as the first proof that the pipeline itself is platform-agnostic — not something rebuilt per integration.